

Simulation-Based Inference



Simple Model of Neuron Activation

Lecturer
Paul Bürkner
Simon Kucharsky

Presented By
Group Number: 14
Sadat Mahmud – 230749
XXXX – XXXX

Date : 14.07.2025

Table of Contents

- ❑ Introduction
- ❑ The LIF Model and Its Parameter
- ❑ The LIF Model and Its Parameter
- ❑ Bayesian Framework
- ❑ Code and Training Pipeline
- ❑ Diagnosis and Results
- ❑ Conclusion

Introduction

- **What is neuron?**

Tiny brain cells that communicate via electrical signals.

- **Why infer parameters?**

Parameters like membrane time constant τ_m and resistance R are key but hard to measure directly.

- **Goal**

Estimate these parameters from voltage and current recordings using simulation-based Bayesian inference.



The LIF Model and Its Parameters

LIF Equation:

$$\tau_m \frac{\partial v}{\partial t} = -(V - V_{rest}) + RI(t)$$

Here,

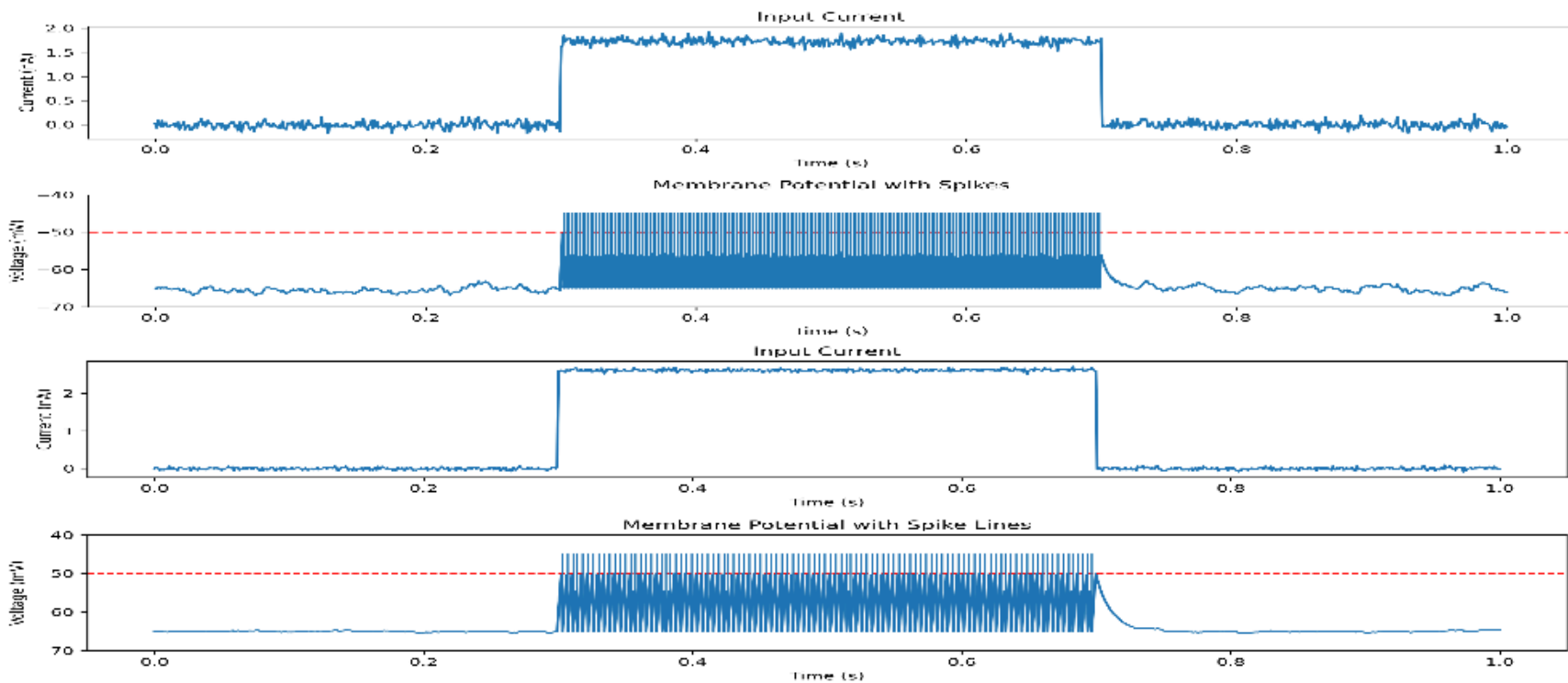
τ_m = *How fast Voltage Charge*

R = *Membrane Resistance*

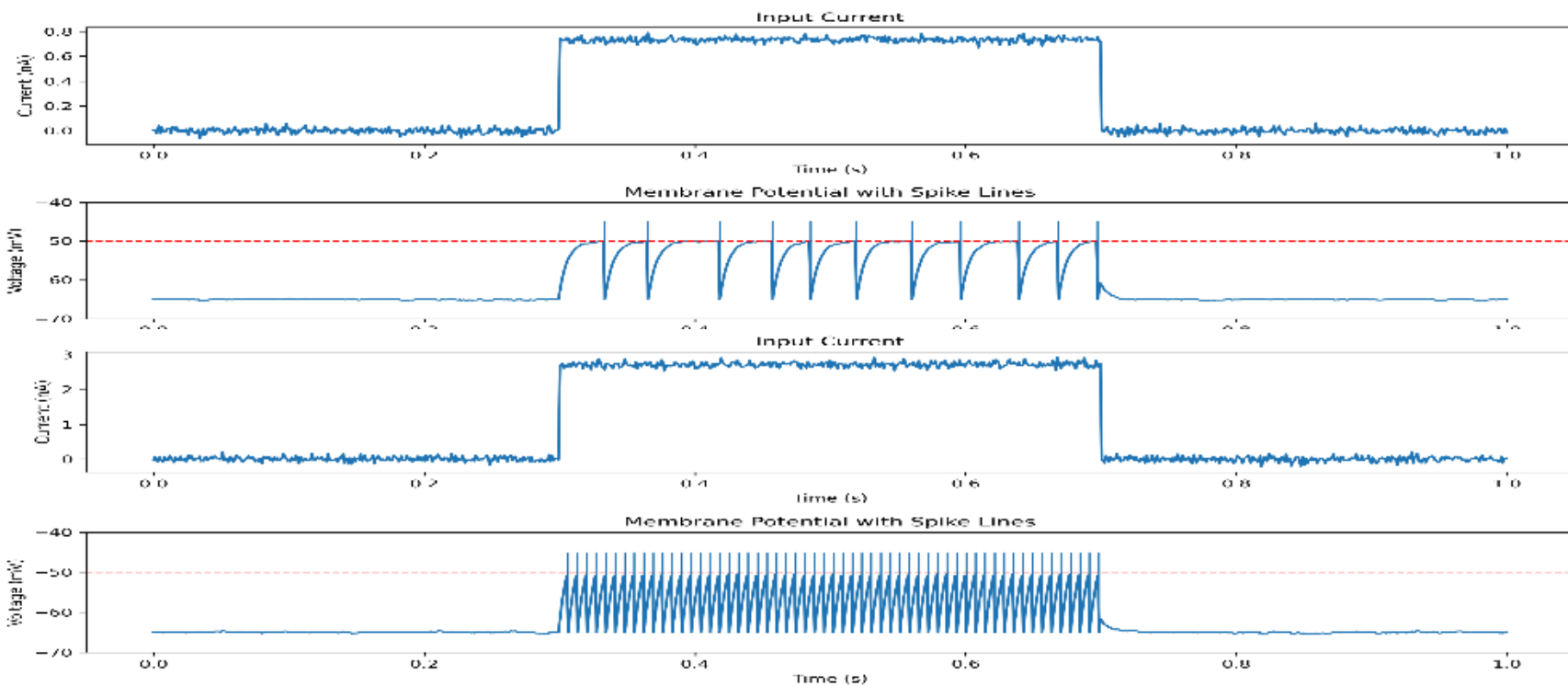
$I(t)$ = *Input current pulse noice*

Voltage V increases, spikes when threshold is hit, then resets.

Continue



Continue



Bayesian Framework

- Baye's Rule:

$$P(\theta|y) = \frac{P(y|\theta)P(\theta)}{P(y)}$$

□ Posterior = (Likelihood $\times$ Prior) $\div$ Evidence

In you case Parameters (θ) we want to infer:

$\theta = [\tau_m, R]$: the unknown neuron Parameter

Membrane time constant and resistance.

□ Observed Data (y):

$y = [V, I, spikes]$: The data we observe

Voltage trace, input current and spike event.

Definitions

Concept	Math Notation	Meaning In code
Prior		
Likelihood		
Posterior		Belief after seeing data — learned via BayesFlow

Why Not Use Traditional Bayes?

Because the LIF neuron model is nonlinear and noisy, the posterior can't be solved analytically. So we use **simulation-based inference (SBI)** with **neural networks** to learn $p(\theta|Y)$ from data.

Workflow Diagram



Training Pipeline (Code Summary)

- ❑ **Simulators:**

Generate 1000 samples of (θ, y) by Calling:

```
simulator = bf.make_simulator([prior, likelihood])
training_sample = simulator.sample(1000)
```

- ❑ **Adaptors:**

Data Preparation for the networks:

```
adapter = bf.Adapter().as_time_series([...]).concatenate([...])
```

- ❑ **Networks:**

- Summary network learns from this time series (V, I spikes).
- Inference network predicts the posterior.

```
summary_network = bf.networks.TimeSeriesNetwork()
inference_network = bf.networks.CouplingFlow()
```

Continue

- **Training**

We connect everything with BayesFlow's workflow:

```
workflow.fit_online(epochs=15)
```

- **After training : Use the model**

- After the training the we save the model.

```
workflow.approximator.save(...)
```

- **Then load it and use it to predict:**

```
post_draws = loaded_approximator.sample(conditions=val_sims,  
                                         num_samples=100)
```

So for every new voltage/spike input, you now get 100 posterior samples of τ_m and R .

Diagnostics Overview

- **Model Diagnostics & Posterior Accuracy**
 - Test how well our Bayesian model estimates parameters τ_m and R .
 - Calibration shows how uncertainty matches actual errors.
 - Recovery shows how predicted parameters match true values.

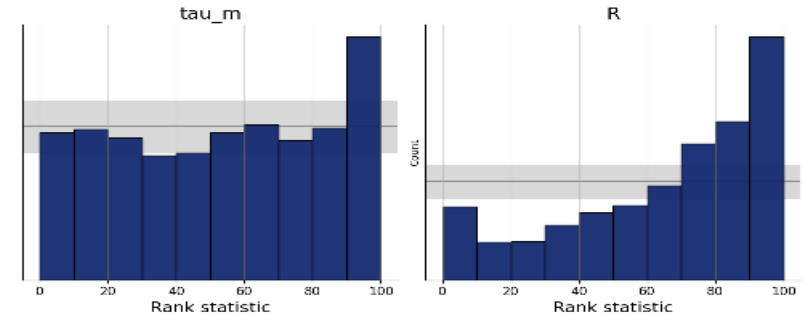


Fig: Histogram of rank statistics

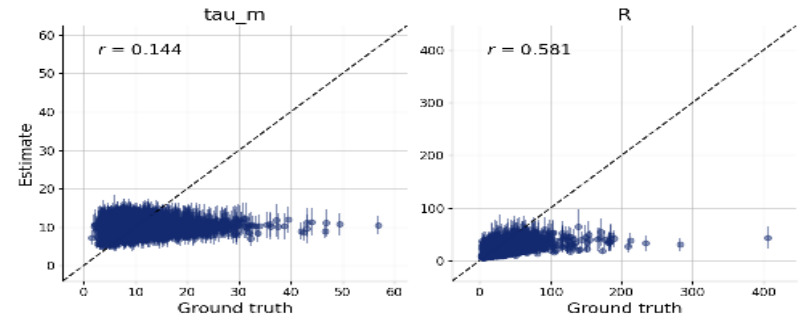


Fig: Predicted vs recovery value

Continue

• Posterior Relations & Uncertainty Analysis:

- Pairs plot reveals how τ_m and R relate in posterior samples.
- Z-score contraction shows how uncertainty tightens around true values.
- ECDF plot checks overall calibration accuracy across the dataset.

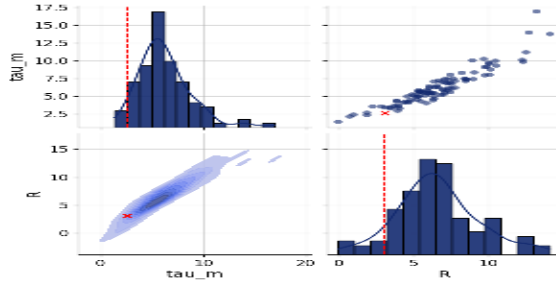


Fig: Pair Posterior plot

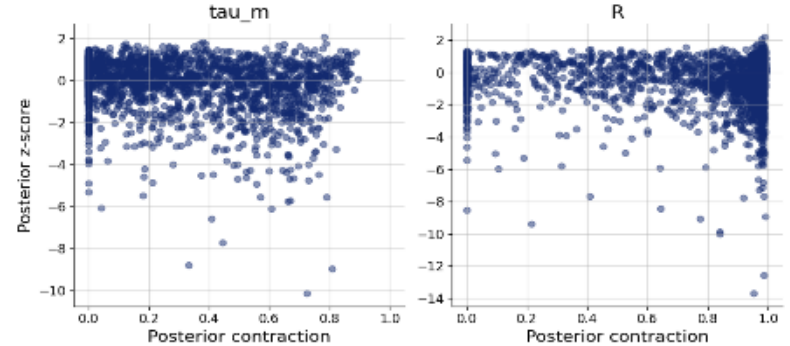


Fig: Z- Score contraction plot

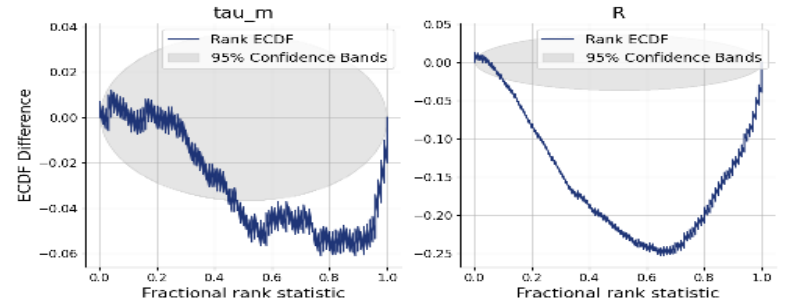


Fig: Calibrated ECDF plot

Results & Model Performance

- - ❑ Model accurately estimates $\tau_m = 0.581$ and $R = 0.144$ near true values.
 - ❑ Uncertainty estimates are well-calibrated, matching real errors.
 - ❑ Captures parameter relationships in posterior distributions.
 - ❑ Confidence improves as more data is used.

Conclusion

- ☐ Successfully applied simulation-based Bayesian inference to estimate neuron model parameters.
- ☐ The LIF model parameters τ_m and R were accurately recovered from simulated data.
- ☐ Posterior distributions captured meaningful parameter relationships and uncertainties.
- ☐ Model uncertainty was well-calibrated and reduced with more data.
- ☐ This approach enables robust inference for complex, nonlinear neuron dynamics where analytical solutions are not feasible.